

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
```

```
file_path = "/content/drive/MyDrive/ML_Project/Group03_Task2_DataSheet.xlsx"

df = pd.read_excel(file_path)

print("Dataset loaded successfully!")
print("Shape:", df.shape)
df.head()
```

Dataset loaded successfully!
Shape: (70, 24)

	Sample No.	tanδ @ 0.000100009 Hz	tanδ @ 0.00021532 Hz	tanδ @ 0.00046404 Hz	tanδ @ 0.00100009 Hz	tanδ @ 0.0021543 Hz	tanδ @ 0.0046415 Hz	tanδ @ 0.01 Hz	tanδ @ 0.021544 Hz
0	1	0.313096	0.263748	0.216453	0.188339	0.194921	0.264037	0.428750	0.76184
1	2	0.308735	0.255590	0.200867	0.157484	0.132245	0.135262	0.165063	0.22867
2	3	0.307918	0.254060	0.197944	0.151697	0.120486	0.111080	0.115364	0.12673
3	4	0.571760	0.423162	0.330518	0.282384	0.273405	0.324580	0.477885	0.81840
4	5	0.565407	0.412576	0.311466	0.246326	0.203263	0.185103	0.197077	0.25529

5 rows × 24 columns

```
print(df.columns)
```

```
Index(['Sample No.', 'tanδ @ 0.000100009 Hz', 'tanδ @ 0.00021532 Hz',
      'tanδ @ 0.00046404 Hz', 'tanδ @ 0.00100009 Hz', 'tanδ @ 0.0021543 Hz',
      'tanδ @ 0.0046415 Hz', 'tanδ @ 0.01 Hz', 'tanδ @ 0.021544 Hz',
      'tanδ @ 0.046416 Hz', 'tanδ @ 0.1 Hz', 'tanδ @ 0.21544 Hz',
      'tanδ @ 0.46417 Hz', 'tanδ @ 1 Hz', 'tanδ @ 2.1546 Hz',
      'tanδ @ 4.6417 Hz', 'tanδ @ 10 Hz', 'tanδ @ 21.563 Hz',
      'tanδ @ 46.512 Hz', 'tanδ @ 100 Hz', 'tanδ @ 216.22 Hz',
      'tanδ @ 470.59 Hz', 'tanδ @ 1000 Hz', 'Class'],
      dtype='object')
```

```
# Column 1 = sample number
sample_no = df.iloc[:, 0]

# Columns 2 to 23 = 22 FDS features
X = df.iloc[:, 1:23].values

# Class column
y = df["Class"].values

print("X shape:", X.shape)
print("y shape:", y.shape)
```

```
print("\nClass distribution:")
print(pd.Series(y).value_counts())
```

```
X shape: (70, 22)
y shape: (70,)
```

```
Class distribution:
Wet          24
Dry          23
Intermediate 23
Name: count, dtype: int64
```

```
X_train, X_test, y_train, y_test, sample_train, sample_test = train_test_split(
    X,
    y,
    sample_no,
    test_size=0.20,
    random_state=42,
    stratify=y
)
```

```
print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
```

```
Training samples: 56
Testing samples: 14
```

```
X_train_log = np.log10(X_train)
X_test_log = np.log10(X_test)

print("Log transformation completed.")
```

```
Log transformation completed.
```

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train_log)
X_test_scaled = scaler.transform(X_test_log)

print("Standardization completed.")
```

```
Standardization completed.
```

```
pca = PCA(n_components=2)

X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

explained = pca.explained_variance_ratio_ * 100

print("PCA completed.")
print(f"PC1 explained variance: {explained[0]:.2f}%")
print(f"PC2 explained variance: {explained[1]:.2f}%")
print(f"Total variance using PC1 and PC2: {explained.sum():.2f}%")
```

```
PCA completed.
PC1 explained variance: 62.04%
PC2 explained variance: 29.31%
Total variance using PC1 and PC2: 91.36%
```

```
classes = ["Dry", "Intermediate", "Wet"]
colors = {
    "Dry": "tab:blue",
    "Intermediate": "tab:orange",
    "Wet": "tab:green"
```

```

}

plt.figure(figsize=(8, 6))

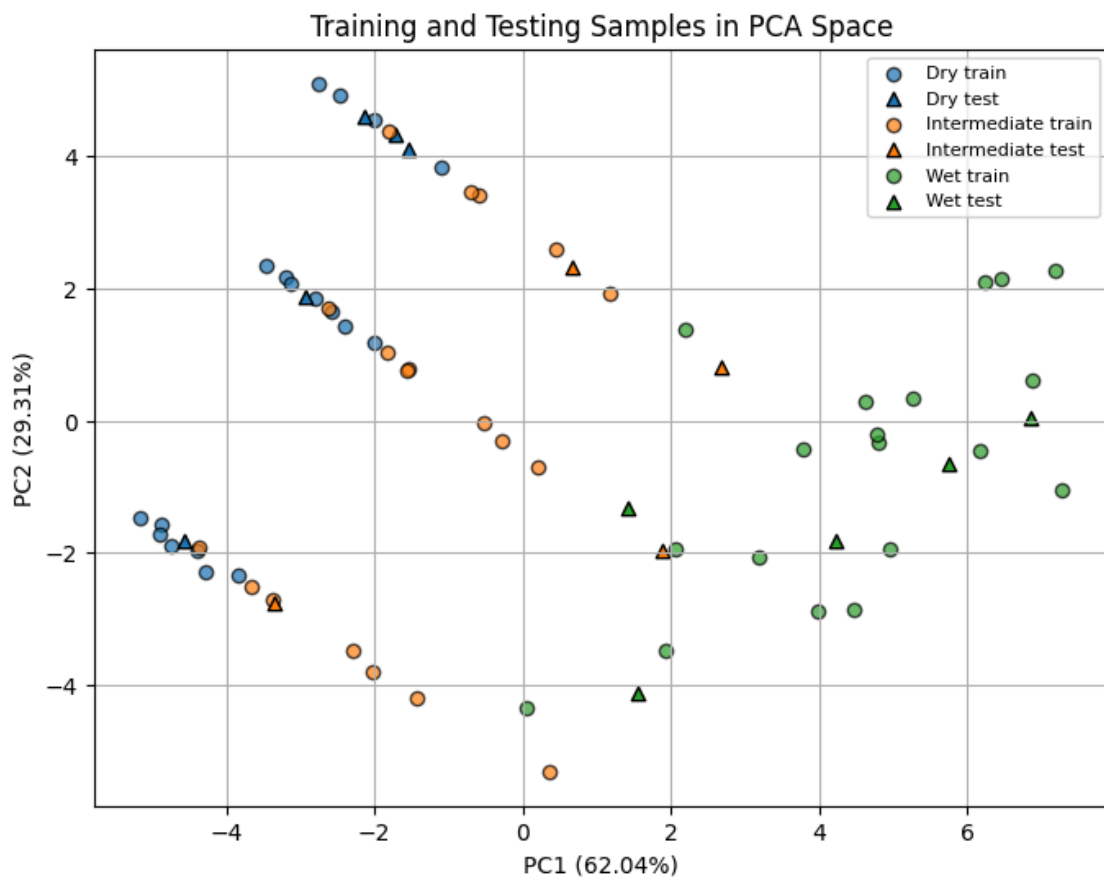
for cls in classes:
    train_idx = y_train == cls
    test_idx = y_test == cls

    plt.scatter(
        X_train_pca[train_idx, 0],
        X_train_pca[train_idx, 1],
        label=f"{cls} train",
        color=colors[cls],
        marker="o",
        edgecolor="black",
        alpha=0.7
    )

    plt.scatter(
        X_test_pca[test_idx, 0],
        X_test_pca[test_idx, 1],
        label=f"{cls} test",
        color=colors[cls],
        marker="^",
        edgecolor="black",
        alpha=1.0
    )

plt.xlabel(f"PC1 ({explained[0]:.2f}%)")
plt.ylabel(f"PC2 ({explained[1]:.2f}%)")
plt.title("Training and Testing Samples in PCA Space")
plt.grid(True)
plt.legend(fontsize=8)
plt.show()

```



```

svm_model = SVC(
    kernel="linear",

```

```

C=1.0,
decision_function_shape="ovr",
random_state=42
)

svm_model.fit(X_train_pca, y_train)

print("SVM model trained successfully!")

```

SVM model trained successfully!

```

y_train_pred = svm_model.predict(X_train_pca)
y_test_pred = svm_model.predict(X_test_pca)

train_accuracy = accuracy_score(y_train, y_train_pred)
test_accuracy = accuracy_score(y_test, y_test_pred)

print(f"Training accuracy: {train_accuracy * 100:.2f}%")
print(f"Testing accuracy: {test_accuracy * 100:.2f}%")

```

Training accuracy: 89.29%
Testing accuracy: 85.71%

```

print("Classification Report:")
print(classification_report(y_test, y_test_pred, labels=classes))

```

Classification Report:				
	precision	recall	f1-score	support
Dry	1.00	1.00	1.00	5
Intermediate	1.00	0.50	0.67	4
Wet	0.71	1.00	0.83	5
accuracy			0.86	14
macro avg	0.90	0.83	0.83	14
weighted avg	0.90	0.86	0.85	14

```

cm = confusion_matrix(y_test, y_test_pred, labels=classes)

plt.figure(figsize=(6, 5))
plt.imshow(cm, cmap="Blues")

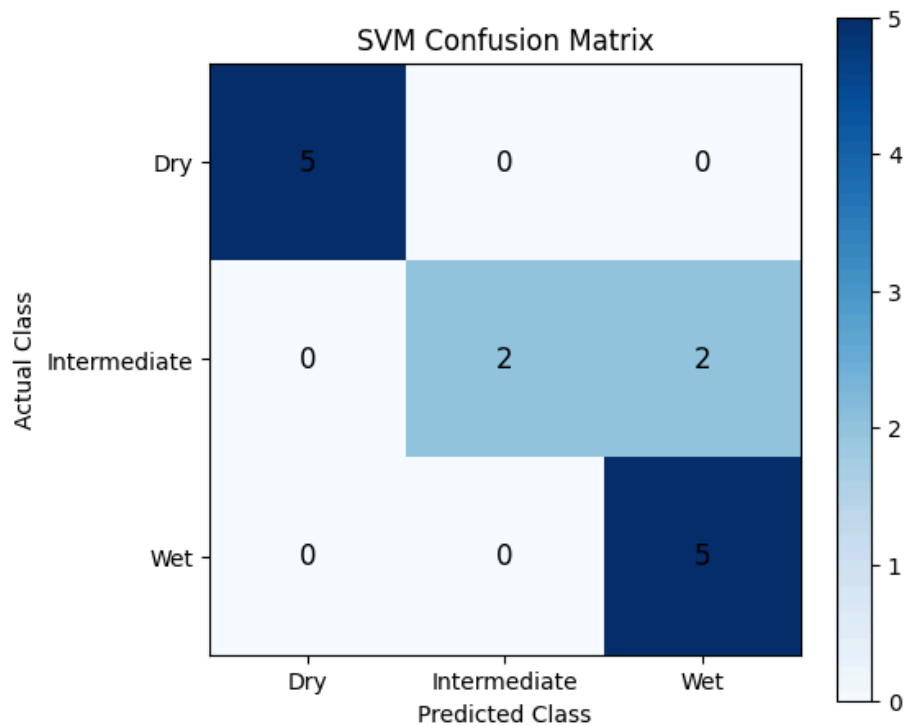
plt.title("SVM Confusion Matrix")
plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")

plt.xticks(np.arange(len(classes)), classes)
plt.yticks(np.arange(len(classes)), classes)

for i in range(len(classes)):
    for j in range(len(classes)):
        plt.text(j, i, cm[i, j], ha="center", va="center", fontsize=12)

plt.colorbar()
plt.tight_layout()
plt.show()

```



```

from matplotlib.colors import ListedColormap

x_min, x_max = X_train_pca[:, 0].min() - 1, X_train_pca[:, 0].max() + 1
y_min, y_max = X_train_pca[:, 1].min() - 1, X_train_pca[:, 1].max() + 1

xx, yy = np.meshgrid(
    np.linspace(x_min, x_max, 400),
    np.linspace(y_min, y_max, 400)
)

grid_points = np.c_[xx.ravel(), yy.ravel()]
grid_pred = svm_model.predict(grid_points)

class_to_num = {"Dry": 0, "Intermediate": 1, "Wet": 2}
Z = np.array([class_to_num[label] for label in grid_pred])
Z = Z.reshape(xx.shape)

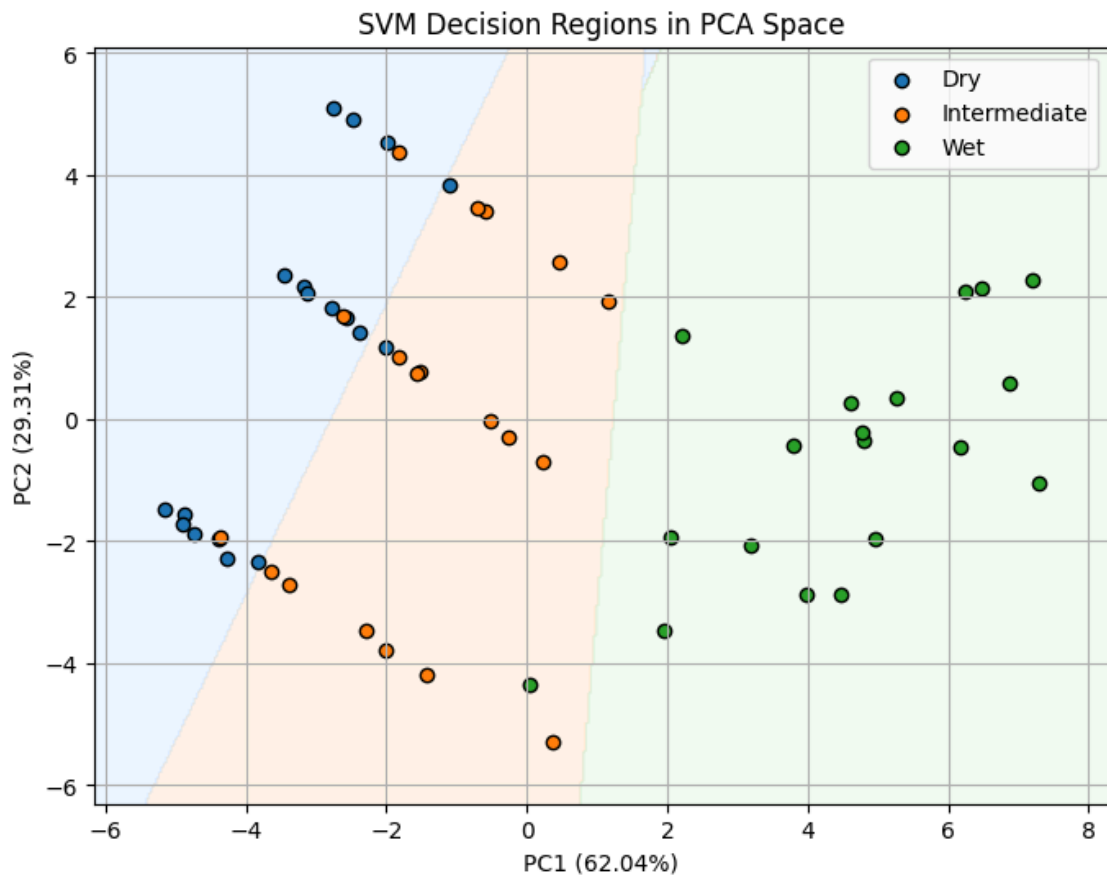
cmap_light = ListedColormap(["#cfe8ff", "#ffe0c2", "#d9f2d9"])

plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, Z, alpha=0.35, cmap=cmap_light)

for cls in classes:
    idx = y_train == cls
    plt.scatter(
        X_train_pca[idx, 0],
        X_train_pca[idx, 1],
        label=cls,
        color=colors[cls],
        edgecolor="black"
    )

plt.xlabel(f"PC1 ({explained[0]:.2f}%)")
plt.ylabel(f"PC2 ({explained[1]:.2f}%)")
plt.title("SVM Decision Regions in PCA Space")
plt.grid(True)
plt.legend()
plt.show()

```



```
svm_all_features = SVC(
    kernel="linear",
    C=1.0,
    decision_function_shape="ovr",
    random_state=42
)

svm_all_features.fit(X_train_scaled, y_train)

print("SVM model trained using all 22 FDS features!")
```

SVM model trained using all 22 FDS features!

```
from sklearn.model_selection import GridSearchCV, StratifiedKFold

param_grid = [
    {
        "kernel": ["linear"],
        "C": [0.01, 0.1, 1, 10, 100],
        "class_weight": [None, "balanced"]
    },
    {
        "kernel": ["rbf"],
        "C": [0.1, 1, 10, 100],
        "gamma": ["scale", "auto", 0.001, 0.01, 0.1, 1],
        "class_weight": [None, "balanced"]
    }
]

cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
    random_state=42
)

grid_search = GridSearchCV(
```

```

estimator=SVC(decision_function_shape="ovr", random_state=42),
param_grid=param_grid,
scoring="accuracy",
cv=cv,
n_jobs=-1
)

grid_search.fit(X_train_scaled, y_train)

print("Best parameters:")
print(grid_search.best_params_)

print(f"Best cross-validation accuracy: {grid_search.best_score_ * 100:.2f}%")

```

```

Best parameters:
{'C': 100, 'class_weight': None, 'kernel': 'linear'}
Best cross-validation accuracy: 96.52%

```

```

best_svm_all = grid_search.best_estimator_

y_train_pred_best = best_svm_all.predict(X_train_scaled)
y_test_pred_best = best_svm_all.predict(X_test_scaled)

train_accuracy_best = accuracy_score(y_train, y_train_pred_best)
test_accuracy_best = accuracy_score(y_test, y_test_pred_best)

print(f"Training accuracy using tuned all-feature SVM: {train_accuracy_best * 100:.2f}%")
print(f"Testing accuracy using tuned all-feature SVM : {test_accuracy_best * 100:.2f}%")

print("\nClassification Report:")
print(classification_report(y_test, y_test_pred_best, labels=classes))

```

```

Training accuracy using tuned all-feature SVM: 100.00%
Testing accuracy using tuned all-feature SVM : 92.86%

```

```

Classification Report:

```

	precision	recall	f1-score	support
Dry	1.00	1.00	1.00	5
Intermediate	1.00	0.75	0.86	4
Wet	0.83	1.00	0.91	5
accuracy			0.93	14
macro avg	0.94	0.92	0.92	14
weighted avg	0.94	0.93	0.93	14